

ServiceNow GlideUser Manual

Comprehensive Reference Guide for Client & Server APIs

The GlideUser API represents the logged-in session context, allowing you to build dynamic form layouts and secure databases. This guide covers how to read variables and run permissions checks on client and server scopes.

1. Client-Side vs. Server-Side User Objects

ServiceNow splits JavaScript execution between client-side scripts (runs in browser) and server-side scripts (runs directly on the cloud server database).

Feature	Client-Side (g_user)	Server-Side (gs.getUser())
Access Syntax	Direct global reference: g_user	GlideSystem API: gs.getUser()
Admin Bypass	g_user.hasRole() bypasses check for Admins.	Must check manually or parse lists on server.
Security Context	UI-only convenience checks (untrusted browser).	Secure database filters and rules (trusted environment).
Email Retrieval	Not supported. Requires client GlideAjax calls.	Direct profile call: gs.getUser().getEmail()
Group Membership	Not supported. Requires client GlideAjax calls.	Direct call: gs.getUser().isMemberOf('Service Desk')
My Group IDs List	Not supported. Requires client GlideAjax calls.	Direct call: gs.getUser().getMyGroups()

2. Client-Side GlideUser (g_user)

The g_user object is pre-instantiated in all client scripts (onLoad, onChange, onSubmit) and UI policies. It operates within browser memory to provide quick checks without lagging UI interactions.

A. Properties

Properties store values describing the logged-in profile:

- g_user.userID: The unique sys_id of the user record.
- g_user.userName: The login username (e.g. 'admin', 'rupam.user').
- g_user.firstName: First name string.
- g_user.lastName: Last name string.

```
// Outputting properties to browser developer tools console
jslog('Logged in User Sys ID: ' + g_user.userID);
jslog('Username: ' + g_user.userName);
jslog('First Name: ' + g_user.firstName);
```

B. Methods

1. getFullName()

Combines first name and last name into a single display string.

```
function onLoad() {
    var fullName = g_user.getFullName();
    g_form.addInfoMessage('Hello ' + fullName + '! Please review form notes.');
```

2. hasRole(roleName)

Returns true if the user possesses the role. Admins automatically return true for this check.

```
if (g_user.hasRole('itil')) {
    g_form.setDisplay('u_internal_work_notes', true);
}
```

3. hasRoleExactly(roleName)

Returns true ONLY if the user has that specific role explicitly assigned. It does NOT automatically return true for Admins.

```
if (g_user.hasRoleExactly('security_admin')) {
    g_form.setReadOnly('u_secure_token', false);
} else {
    g_form.setReadOnly('u_secure_token', true);
}
```

3. Server-Side GlideUser (gs.getUser())

On the server (Business Rules, Script Includes, Workflow Scripts), the user context is retrieved calling `gs.getUser()`. This object accesses the database directly, offering rich methods.

A. Profile Retrieval Methods

Fetch details off the database profile card:

- `gs.getUserID()`: Shorthand returning the user's primary `sys_id`.
- `gs.getUserName()`: Shorthand returning user login identifier.
- `gs.getUser().getDisplay_name()`: Full display name.
- `gs.getUser().getEmail()`: Email address.
- `gs.getUser().getDepartmentID()`: Department record `sys_id`.
- `gs.getUser().getCompanyID()`: Company record `sys_id`.
- `gs.getUser().getManagerID()`: Manager user `sys_id`.

```
var user = gs.getUser();
gs.info('User Display Name: ' + user.displayName());
gs.info('User Email: ' + user.getEmail());
gs.info('User Company: ' + user.getCompanyID());
```

B. Group Membership Checks

1. isMemberOf(group)

Returns true if the user is a member of the group. Accepts the Group Name or its `sys_id`.

```
// Business Rule Example: Assign to Service Desk if caller is a member
if (gs.getUser().isMemberOf('Service Desk')) {
    current.setValue('assignment_group', 'sys_id_of_service_desk');
}
```

2. getMyGroups()

Returns an `ArrayList` containing the Sys IDs of all groups the current user belongs to. Useful for filtering query databases.

```
// Query incidents assigned to any group the current logged-in user belongs to
var userGroups = gs.getUser().getMyGroups();
var incGR = new GlideRecord('incident');
incGR.addQuery('assignment_group', 'IN', userGroups);
incGR.query();
while (incGR.next()) {
    gs.info('Incident assigned to my group: ' + incGR.getValue('number'));
}
```

4. Security Design: The Client-Side Fallacy

Client-side code runs inside the user's browser, which means it is in an untrusted environment. Technical users can press F12, open the Javascript Console, and bypass your client-side rules (e.g. running `g_form.setReadOnly()` or modifying `g_user` checks).

SECURITY WARNING: Do not use Client Scripts (`g_user`) as your only line of defense for locking down fields, forcing validations, or hiding secure tabs.

The Golden Rule of ServiceNow Security:

- Client-Side (`g_user` / Client Scripts): Use to improve User Experience (UX) by dynamically layout forms, autofilling details, and showing alerts.
- Server-Side (`gs.getUser()` / Business Rules & ACLs): Enforce database transactions. Make sure the database rejects unauthorized operations if a user bypasses the UI.

Secure Design Example (Access Control List - ACL script):

Create a write ACL on the Secure Field table to check groups on the server:

```
// Server-Side ACL Script (Runs securely on instance instance)
// Allow write access only if the user is a member of the 'Security Managers' group
if (gs.getUser().isMemberOf('Security Managers')) {
    answer = true;
} else {
    answer = false;
}
```